

Semi Global Block Matching Acceleration

Vasilis Pteris¹, Alexandra Ukrainskaya², Ishan Gupta³, Asavri Kaur⁴
School of Computer Science and Engineering
The University of New South Wales
Sydney, Australia

¹z5418252, 5418252@ad.unsw.edu.au
³z5620669, z5620669@ad.unsw.edu.au

²z5470334, z5470334@ad.unsw.edu.au
⁴z5627841, z5627841@ad.unsw.edu.au

I. BACKGROUND

Stereo depth estimation involves determining the depth of a 3D scene from a pair of horizontally rectified images from two different cameras, and finding the corresponding pixels between the left and right images and computing their relative offset, or disparity.

Many algorithms exist for computing disparity between stereo images, with various trade-offs between accuracy, latency and computational cost. There are local matching methods, like block matching, that are cheap computationally, but create noise in their disparity maps. Global methods produce highly accurate results as an energy minimization problem over a full image, but at such a high latency that they are impractical for real use.

Semi-Global Matching (SGM) is an algorithm introduced by Hirschmuller [1] that aimed to achieve a compromise between the two ends of the spectrum. Rather than optimizing a single global energy function over an entire image (computationally heavy), SGM approximates by aggregating matching costs along multiple 1D paths that traverse the image from different directions and combining the results to select disparity for each pixel. This approach retains the accuracy of global methods while keeping computational load much more manageable. Although SGM's efficiency compared to global methods is significantly better, the cost still makes real-time performance impractical on general purpose CPUs. This is the motivation behind using hardware accelerated SGM-based methods on FPGAs. This project explores such an acceleration approach using Vitis HLS, the Vitis Vision SGBM library and the Kria KV260 platform.

A. SGM/SGBM Algorithm

The SGBM algorithm has 3 main stages: cost computation, cost aggregation and disparity selection, as shown in Fig. 1.

Cost computation computes a matching cost for each pixel in the reference image against candidate pixels in the target image across a range of matching disparity values. This matching cost quantifies how well a given disparity explains the observed pixel intensities, typically by using a pixel dissimilarity measure like mutual information, absolute difference or census transform.

Cost Aggregation aggregates costs along multiple 1D paths, radiating outwards, per pixel, in typically 8 or 16 directions to incorporate the influence of neighboring pixels. On each path the aggregated cost at a pixel and disparity is computed as the pixel's own matching cost, plus the minimum of the previous pixel's aggregated cost over a small window of nearby disparities. There is also a small penalty for small disparity changes to encourage smooth surfaces and a large penalty over large disparity changes to allow object edges to exist.

Disparity Selection chooses the the disparity with the lowest aggregated cost as the final estimate, per pixel. Usually this is followed by sub-pixel interpolation for increased precision and post-processing steps like left-right consistency checking to identify and remove unreliable matches.

II. RESEARCH GAP

A. Existing SGBM Implementations

There are a lot of existing hardware and software implementations of SGBM. The industry-standard software implementation is `cv::StereoSGBM` by openCV.

It is an SGBM implementation that is based on the one described by Hirschmuller [1], but has a few differences. Namely, it only considers five directions by default during cost aggregation stages; it matches blocks rather than individual pixels; and it replaces mutual information with cheaper Birchfield-Tomasi sub-pixel metric. It also includes pre-processing and post-processing stages, including uniqueness check, quadratic interpolation, and speckle filtering. [2]

The opencv's implementation allows the user to tune the algorithm's parameters. The search range is set by `minDisparity` and `numDisparities`, the latter constrained to multiples of 16 by the vectorised memory layout. Cost computation is controlled by `blockSize`, which determines the box-filter window applied to per-pixel costs, and `preFilterCap`, which truncates the horizontal gradient before it reaches the Birchfield-Tomasi function. Smoothness depends on the penalties `P1` and `P2`, applied respectively to unit and larger disparity changes along each aggregation path. The remaining parameters configure post-processing: `uniquenessRatio` rejects matches whose best cost fails to beat the runner-up by a given margin, `disp12MaxDiff` bounds the permitted discrepancy in the left-right consistency check, and `speckleWindowSize` together with `speckleRange` remove small isolated disparity regions in the output. [2]

Besides software implementations, there have been hardware implementations on FPGA. Most of those were implemented in Hardware Description Languages Verilog or VHDL. [3] The canonical example is the systolic-array architecture of Banz et al., which sustains 640×480 at 30 fps with a 128-pixel disparity range on a Xilinx Virtex-5 clocked at only 39 MHz. [4] More recent work targets the same device family as our platform: Kalomiros et al. report a throughput of 55.1 million disparity estimates per second at 1280×720 with 128 disparities on a Zynq UltraScale+ device. [5]

HDL solutions have been able to achieve high throughput, they still tend to outperform HLS solutions in terms of throughput in general [6] and in semi-global matching specifically [3]. However, compared to HLS designs, they are more difficult to reproduce or modify. [3]

In this project, we based our optimization on the HLS implementation by Xilinx from Vitis Vision library. Unlike the implementation by opencv, this implementation matches individual pixels rather than blocks. It doesn't include any post-processing or pre-processing, unlike its opencv counterpart. The implementation in Vitis Libraries allows the user to set the dimensions of the input, number of disparities, and the number of processing units. [7]

The number of processing units parameters determines how many disparities are evaluated in parallel per clock cycle - if it is set equal to the disparity range (`PU == NDISP`), the pipeline resolves one pixel's full disparity range per cycle at an initiation interval of one; otherwise it iterates `NDISP/PU` times per pixel - this highlights the trade-off between latency and resource consumption. [7]

Another HLS implementation of SGBM algorithm is FP-stereo, which targets the same Zynq UltraScale+ class of device as our platform (evaluated on the ZCU102). We have tried running on Kria board to compare it to our improved implementation, but we couldn't because it relies on outdated SDSoc tool that aren't supported anymore. Another difficulty is that ZCU102 board has significantly more resources than Kria board. [3], [8], [9]

Unlike the Xilinx's implementation, FP-Stereo is a framework that allows to create custom pipelines rather than a fixed solution - for example, it allows the user to choose between several different computation functions. To maximize hardware efficiency, FP-Stereo intelligently packs multiple data values into single memory words, significantly reducing

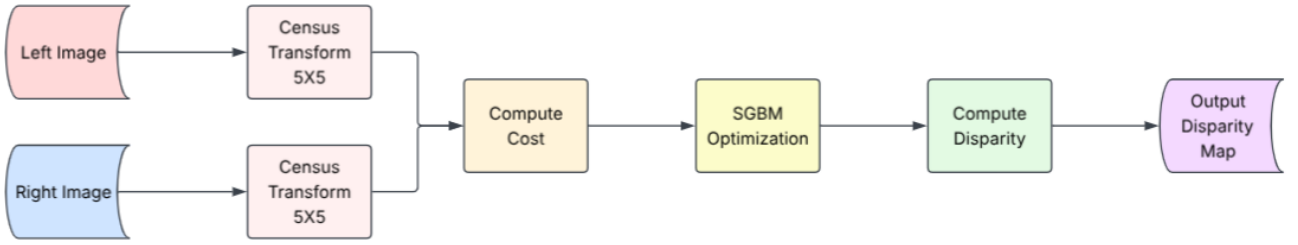


Fig. 1. Overview of the SGBM pipeline from stereo images to the output disparity map.

on-chip memory consumption compared to the standard Xilinx approach. Furthermore, FP-Stereo resolves pipeline stalling issues by reorganizing the order in which pixels are processed. This interleaving technique ensures a continuous, uninterrupted flow of calculations, allowing FP-Stereo to achieve higher processing speeds and lower energy usage than the baseline Xilinx solution. [3]

According to the authors of FP-Stereo paper, FP-Stereo offers a solution that has 6.08% lower error rate than the one by Xilinx, while being twice as fast. [3] However, since FP-Stereo was evaluated on a different board on different data, we can't compare the results from the paper to ours.

The Vitis Vision library target devices ranging from Zynq-7000 to Alveo, but its published verification covers the zcu102, zcu104, vck190, U50, and U200 boards. [7]; Kria-class SOMs are not among them. The same holds for the literature. To the best of our knowledge, no prior work characterises an SGBM accelerator on a Kria-class device.

The library implementation is also known to use its resources inefficiently: at matched algorithmic parameters, FP-Stereo recovers enough on-chip memory and pipeline throughput to run twice as fast on the ZCU102 [3]. FP-Stereo directs that recovered budget toward speed. On a device where memory, rather than throughput, is the binding constraint, the same savings could instead be spent on the accuracy of the disparity map - an option the literature on HLS acceleration of SGBM has not explored.

This work does both. We profile the unmodified Vitis Vision design on the KV260 and identify the on-chip memory bottleneck that prevents the disparity range from being increased. We then restructure the pipeline to reduce that consumption and reinvest the recovered resources in accuracy: higher input resolution, sub-pixel refinement, and a modified cost aggregation scheme, none of which are feasible in the baseline configuration on this device.

B. Related Work

Besides Semi-Global Block Matching, there are other disparity map methods. There are local block matching methods, which are faster but produce lower-quality disparity maps, and global matching methods, which produce better-quality disparity maps, but have significantly higher latency.

Stereo local matching algorithms follow the same cost computation, cost aggregation, and disparity selection stages described for SGBM in the introduction section, but each stage is simplified to operate independently per pixel. Cost aggregation sums or averages the matching cost over a fixed support window rather than along multiple directional paths, and disparity selection reduces to a local winner-takes-all (WTA) rule that assigns each pixel the disparity with the lowest aggregated cost, without SGBM's inter-pixel smoothness penalties. [10] This omission of cross-pixel regularization is what makes local methods faster than SGBM, but also what produces the noisier disparity maps described above, particularly in textureless or repetitive regions. [10]

In contrast, global matching algorithms replace SGBM's per-direction cost aggregation with a single energy-minimization problem solved jointly over the whole image. Global methods skip cost aggregation entirely and instead minimize an energy function. This is done using techniques such as belief propagation, graph cuts, or dynamic programming, all of which require considering the full image simultaneously rather than the local windows or independent 1D paths used by local and semi-global methods. Because global methods take into consideration the whole image, they produce the best disparity maps, but their computational price makes them impractical. [10]

SGBM algorithm, which we optimized in this project, produces better map than local matching methods [4], while having significantly lower computational cost than global methods. [10]

C. SGBM in Aerial Imagery

In this work, we targeted SGBM for aerial imagery and evaluated the results on WHU MVS/Stereo Dataset. [11] Aerial imagery is a natural choice for embedded stereo acceleration. This is because aerial platforms are often constrained in power. High-throughput implementations of semi-global matching are typically implemented on GPUs, but GPUs consume more power than FPGAs. [10]

Compared to close-range images, aerial images tend to have more large homogenous regions. This justifies the choice of semi-global matching over local matching, since local matching is especially bad at producing matches in repetitive regions. [10]

The precision requirement is likewise stricter than in close-range stereo. Depth error scales with the square of range for a fixed disparity error. This motivated introduction of subpixel interpolation to the design.

III. GOALS AND OBJECTIVES

This project underwent several design iterations as our understanding of the Semi-Global Block Matching (SGBM) algorithm and its hardware implementation developed. Our original goal was to implement OpenCV's SGBM algorithm using High-Level Synthesis (HLS). However, the OpenCV implementation contained features that were either beyond the scope of the project or unsuitable for efficient HLS optimisation.

During our research, we identified the Vitis SGBM implementation, which was specifically designed for FPGA-based acceleration, and selected it as our baseline. Since this implementation was already highly optimised, achieving a substantial improvement in raw performance alone proved difficult. We therefore shifted our focus towards improving disparity estimation accuracy while maintaining real-time performance and acceptable FPGA resource utilisation.

Our investigation showed that disparity range and processing parallelism significantly affected accuracy, throughput, and resource utilisation. We also identified opportunities to improve accuracy at different stages of the SGBM pipeline. However, these modifications introduced additional computational and resource requirements, requiring further hardware optimisation.

The primary goals and objectives of the project were to:

- use the Vitis SGBM HLS implementation as the hardware baseline;
- investigate the relationships among disparity range, processing parallelism, accuracy, throughput, and FPGA resource utilisation;
- implement and evaluate modifications intended to improve disparity estimation accuracy;
- optimise the modified design for FPGA execution while maintaining approximately 70% or lower utilisation across the major resource classes, based on FPGA design methodology guidance [12];
- achieve a throughput of at least 24 frames per second, corresponding to a maximum processing time of approximately 41.7 ms per stereo frame;
- develop a host-side pipeline capable of supplying and processing multiple stereo image pairs in batches; and
- validate the generated disparity maps against the WHU MVS/Stereo drone dataset ground-truth disparities [13] using bad-pixel rate, average disparity error, and error percentiles based on the Middlebury evaluation methodology [14], and compare the results with the Vitis Vision baseline.

Overall, the project evolved from a direct acceleration task into an investigation of the trade-offs among disparity accuracy, throughput, parallelism, and FPGA resource utilisation. The final aim was to produce a measurable SGBM pipeline that maintained real-time performance while providing verifiable accuracy improvements.

IV. METHODOLOGY

A. Project Work Breakdown

Phase 1 — Baseline Profiling: The Vitis Vision SGBM pipeline was synthesized as a baseline. It was profiled to measure latency and resource utilization. This profiling identified the main performance bottleneck and established the design constraints, including setting NDISP to 64 and PU to 32. Increasing any of these was found to be infeasible because it exceeded the available KV260 resources and failed during hardware linking.

Phase 2 — Optimizations: the SGBM code was divided into four key functions, with each team member responsible for optimizing one function. This allowed optimization work to proceed in parallel while each function could be evaluated independently. The specific function allocation and contributions of each member are described in Section B.

Phase 3 — Integration, evaluation, and demonstration. The optimized stages were integrated into the complete pipeline and validated against the software reference. The implementation was then extended to process multiple stereo frames rather than a single frame at a time, improving its suitability for video inputs. Software versions of the SGBM pipeline were also implemented on the Kria KV260 to establish an ARM-based performance baseline for measuring the speedup achieved by the FPGA accelerator. Accuracy was evaluated across the implementations by comparing the generated disparity maps against the ground truth provided in the dataset.

B. Vitis Vision Library Function Allocation

The optimisation work was divided across four key stages of the Vitis Vision SGBM pipeline, with each team member responsible for a specific function. This allowed the optimisation of individual stages to be developed and evaluated independently before integration into the final pipeline.

- **Cost Computation — Vasilis:** The Census-based matching cost was extended with an Absolute Difference (AD) cost to improve matching in low-texture regions commonly found in aerial imagery. A LUT-based weighting scheme was implemented to combine the two costs while maintaining the synthesize pipeline structure.
- **Disparity Selection — Alexandra:** The disparity calculation stage was modified to support sub pixel disparity estimation using parabolic interpolation. A bug in the original implementation that produced substantial salt-and-pepper noise was also identified and corrected. Output disparity quality was evaluated against the dataset ground truth.
- **Path Optimization — Asavri:** The path-aggregation stage was optimized by restructuring the frequently accessed L_r buffer and investigating alternative memory architectures. The software implementation of the Vitis baseline was also successfully deployed on the Kria KV260, and the baseline hardware binary container was prepared for demonstration and comparison.
- **Frame Processing and Throughput — Ishan:** The host-side pipeline was extended to process multiple stereo frames, rather than a single image pair, enabling batched processing for video inputs. The Vitis Vision baseline was also deployed and tested on the KV260 hardware.

V. IMPLEMENTATION & RESULTS

A. Optimization Methods

AD-Census Cost Fusion. The baseline Vitis Vision cost computation stage relies solely on the Census transform for pixel matching cost. While Census is robust to radiometric differences between the left and right images, it discards raw intensity information, which can produce ambiguous or unstable matches in low-texture regions, which is a particular concern for the large homogeneous areas common in aerial imagery. To address this, we fused the Census-based Hamming distance cost with a raw pixel Absolute Difference (AD) cost, following the general approach of [15]’s AD-Census. Rather than combining the two cost terms with the

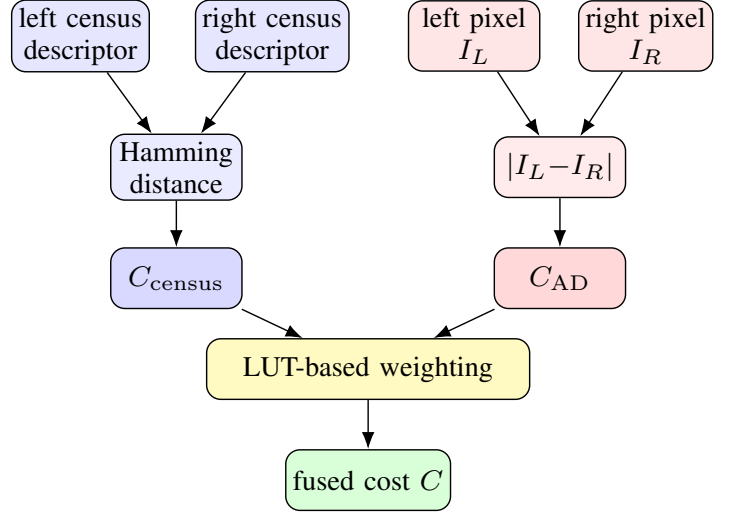


Fig. 2. AD-Census cost fusion. Census-based Hamming distance and absolute-difference (AD) costs are computed in parallel and combined using LUT-based weighting to produce the fused matching cost.

exponential robust functions used in the original AD-Census formulation, which are expensive to implement in fixed-point HLS, we combined them using a lookup-table (LUT) based weighting scheme, replacing the floating-point exponential evaluation with a precomputed piecewise mapping. This preserved the intent of the combination while remaining synthesizable within the II=1 pipeline constraint of the existing cost computation stage. The function $\rho(c, \lambda) = 1 - e^{-c/\lambda}$ was precomputed, and used to bound the influence of outlier costs, as two separate LUTs: a 25-entry table over the Census Hamming-distance domain ($\lambda_{\text{census}} = 10.0$) and a 256-entry table over the AD domain ($\lambda_{\text{AD}} = 30.0$). Both LUTs were scaled to a maximum of 127 rather than 255, so that the fused cost $C = \rho_{\text{census}}[C_{\text{census}}] + \rho_{\text{AD}}[C_{\text{AD}}]$ saturates at 254 and fits within the existing stream width without altering any downstream interfaces in xFSGBMoptimization.

For a candidate disparity, I_L and I_R denote the raw pixel intensities at the corresponding positions in the left and right images, respectively. The absolute difference $|I_L - I_R|$ is used to form the AD cost, C_{AD} , providing intensity information that is not retained by the Census descriptor. In parallel, the 5×5 Census descriptors from the left and right images are compared using their Hamming distance to produce the Census cost, C_{census} . As illustrated in Fig. 2, these two cost terms are computed independently and then passed to the LUT-based weighting stage, which combines them to produce the final fused matching cost C .

Subpixel interpolation. The baseline Vitis implementation disparity selection stage chooses an integral value as an output for each pixel. To increase the accuracy of the estimation and return fractional results, we added subpixel interpolation. Subpixel interpolation uses costs of the lowest-cost disparity and its two nearest neighbours to find a value within $[\text{lowest cost disparity} - 0.5, \text{highest cost disparity} + 0.5]$ range. Specifically, we used subpixel parabolic interpolation.

For each pixel, the disparity d corresponding to the minimum matching cost is first identified. Rather than using this integer disparity directly, we use the costs at the neighboring disparities, C_{d-1} , C_d , and C_{d+1} , to estimate a local quadratic cost curve, as illustrated in Fig. 3. The vertex of this parabola provides the subpixel disparity estimate, given by

$$d_{\text{sub}} = d + \frac{C_{d-1} - C_{d+1}}{2(C_{d-1} - 2C_d + C_{d+1})}.$$

This allows the estimated disparity to lie between the integer disparities rather than being restricted to an integer value. The resulting fractional disparity is represented using a 16-bit output, where the encoded value is divided by 256 to recover the actual disparity, matching the encoding used by the WHU MVS dataset.

The parabola formula used in interpolation contains division operation, which is expensive in terms of LUTs. To solve this problem, we replaced the division with BRAM look-up tables. Since most output values in this application are in the lower range, we created two BRAM look-up tables - one is for denominator values from 0 to 2047 with higher resolution,

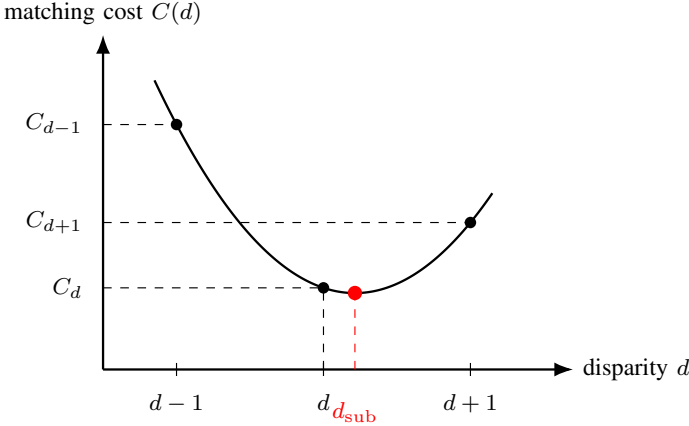


Fig. 3. Parabolic subpixel interpolation using the minimum-cost disparity and its two neighbouring disparities.

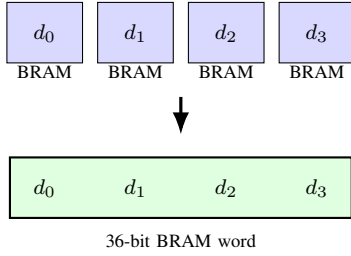


Fig. 4. L_r buffer reorganisation from individual disparity costs to a packed 36-bit BRAM word containing four disparity values.

and the other is for the other possible denominator values with lower resolution.

Memory Architecture.

The L_r buffer stores the path-aggregation costs from the four SGBM directions and is read and updated for every pixel. In each iteration, the algorithm processes a number of disparity costs equal to the configured number of parallel units. In the original implementation, each disparity cost occupied a separate memory word. Providing simultaneous access to all parallel lanes therefore required the array to be partitioned across individual BRAMs, resulting in inefficient use of the available memory width.

To improve memory efficiency, we reorganised the L_r buffer by packing multiple disparity costs into wider memory words. Since each disparity cost required 9 bits, four costs could be packed into a single 36-bit BRAM word, as illustrated in Fig. 4. The number of disparity lanes processed concurrently remained equal to the configured number of parallel units; however, the packed representation allowed four lanes to share the width of one BRAM word instead of storing each lane in a separate word.

This reorganization reduced the number of BRAM instances required and allowed the design to support greater disparity-level parallelism within the available resources. It also reduced the observed latency of the optimization stage, likely because of the lower BRAM count and simpler associated control logic.

Originally, we considered using UltraRAMs because they offer a 72-bit memory width; however, the resulting design failed to meet timing during bitstream generation.

Disparity Range We tried increasing the number of disparities. This changes the range of disparity values that the SGBM algorithm searches when matching pixels between the stereo images. We made this change because our drone dataset contains large depth variations, and the original disparity range was too small to capture all valid matches.

By increasing the disparity range, the algorithm could search over a wider range of disparity values, improving the quality of the generated disparity map. The trade-off was that more disparities must be evaluated for every pixel, increasing the computational workload. This resulted in higher resource utilisation and increased latency, but significantly improved disparity accuracy on our drone dataset. The final implementation used $NDISP = 96$, up from the $NDISP = 64$ established during baseline profiling (Phase 1).

B. Implementation of Batched Processing

The original Vitis SGBM accelerator processed one stereo image pair per kernel invocation using memory-mapped AXI interfaces. To support batch processing, we modified the architecture to process multiple consecutive image pairs within a single invocation. A runtime parameter, `frame_count`, specifies the number of frames in each batch, giving a total pixel count of

$$N_{\text{batch}} = \text{rows} \times \text{cols} \times \text{frame_count}. \quad (1)$$

The direct memory interfaces of the original accelerator were replaced with AXI4-Stream interfaces. Two `video_mm2s_batch` kernels read the left and right image batches from global memory and convert the packed input data into pixel streams. These streams are processed by the modified `semiglobalbm_accel` kernel, after which the `video_s2mm_batch` kernel packs the resulting disparities and writes them back to global memory.

The input images contain 8-bit pixels, with four pixels packed into each input memory word. The memory-to-stream kernels unpack these values and transmit one pixel at a time. Similarly, four 16-bit output disparities are packed into each output memory word. The conversion loops use `PIPELINE II=1` directives to target a processing rate of one pixel per clock cycle.

Frame boundaries are derived from the pixel count. The AXI4-Stream user signal marks the first pixel of each frame, while `last` marks the final pixel of each row. Since each frame contains exactly $\text{rows} \times \text{cols}$ pixels, consecutive frames can be separated without additional padding.

To extend the algorithm across multiple frames, the single-frame `SemiGlobalBM` function was replaced with `SemiGlobalBMBatch`. Batch wrappers invoke the required processing functions once per frame, preserving frame-dependent state while allowing the complete batch to pass through the streaming architecture.

An HLS DATAFLOW region connects the internal processing stages through FIFO streams. The synthesis dataflow report identifies separate processes for the two Census transformations, Census unpacking, cost computation, optimisation, and disparity computation. The corresponding streams connect these processes in sequence, allowing different stages to operate concurrently subject to data availability and FIFO backpressure.

The cost and aggregated-cost stream arrays were completely partitioned across the processing-unit dimension. In the synthesised configuration, this produced 48 independent 8-bit cost streams and 48 independent 16-bit aggregated-cost streams. Each stream was configured with a depth of 32, allowing the parallel units to communicate through separate FIFO channels.

We evaluated several FIFO depths to balance buffering capacity, latency, and FPGA resource utilisation. A depth of 32 maintained the observed latency while avoiding BRAM usage for the reported internal streams, making it the most suitable configuration for our final implementation.

A depth of 64 produced an unexpected increase in latency, while larger depths returned to approximately the expected value. The cause of this non-monotonic behaviour was not conclusively identified, but may relate to memory inference, HLS scheduling, clock period, or dataflow stalling.

C. Unused Approaches

Median Filter During development, there was a pepper-and-salt noise issue. Due to the noise, the output disparity maps had average error of 16.15 pixels, which is worse than the baseline design.

To solve it, we considered adding a Switching Median Filter to the pipeline and tested it on software. The filter is similar to a conventional median filter, but instead of being applied to every pixel, it is only applied to pixels identified as noisy. To determine whether a pixel is corrupted, its value is compared with those of its neighbours within a specified window. We used a window size of 5 and identified a pixel as noisy when its value differed from approximately 70% of its neighbours by more than a set threshold. In the implementation, this threshold was set to 2 pixels. If a pixel is identified as noisy, its value is replaced with the median of its neighbouring pixels within the window. The filter was also applied twice, with the output of the first pass used as the input to the second. This approach was effective at removing salt-and-pepper noise, but the software implementation was too slow to include it into the pipeline.

We considered implementing it on hardware, but we decided to focus on the other parts of the project after identified the bug that caused the noise.

Array Reshape. We also explored using the `ARRAY_RESHAPE` pragma to improve memory access parallelism. However, it did not provide the same performance improvement as explicitly packing four disparity values into a wider 36-bit BRAM word. The explicit packing approach was therefore adopted in the final implementation, while the `ARRAY_RESHAPE` approach was not used.

D. Accuracy Profiling

The most commonly used metrics to evaluate disparity maps are average error - which is an average difference between calculated disparity and actual disparity for each pixel; percent of bad pixels, which is the number of pixels for which the error is greater than some threshold, typically 1 or 2 pixels; and the number of invalid pixels, which is the number of pixels that the algorithm couldn't find disparity for. Since our design doesn't have any validation steps, the number of invalid pixels isn't a relevant metrics for this project.

Initially, we evaluated our results on Middlebury Dataset. To evaluate accuracy of produced disparity maps, we used the Middlebury SDK evaluation tool. The tool evaluates disparity maps in pfm format against the ground truth, which is also in pfm format, and calculates average error and proportion of bad pixels. [14] However, after we decided to focus on aerial imagery, we decided to evaluate our outputs using a simple python script.

The python script reads two png images and compares them pixel-by-pixel and calculates average error, proportion of bad pixels for threshold of 1 pixel, proportion of bad pixels for threshold of 2 pixels, average error, median error, and 90th percentile error. The reason why we opted for a custom python script over a standard evaluation tool is that the output of the host code is in png format, and the ground truth for the drone dataset is in png format, so it is easier to compare them directly instead of converting to .pfm before evaluation.

E. Hardware Profiling

Hardware performance was profiled on the Kria KV260 using the host-side C++ application. The final implementation was evaluated using 48 stereo frames to capture the behaviour of the batched pipeline. Hardware pipeline latency was measured using `std::chrono`, with timing starting immediately before the MM2S, SGBM, and S2MM kernels were launched and ending after all kernels completed. This measurement captures the execution time of the hardware pipeline excluding host-to-device and device-to-host data transfers. End-to-end latency was measured separately, with timing starting before the input buffers were transferred to the device and ending after the output buffer was transferred back to the host. Average latency per frame was calculated by dividing the measured total time by the 48 processed stereo frames. The software-hardware execution flow used for profiling and the final implementation is shown in Figure 5.

F. Quantitative Results

To validate the AD-Census cost fusion independently of the full pipeline, we evaluated it against the Census-only and AD-only cost functions using the Middlebury MiddleEval3 stereo benchmark, using the standard `bad0.5`, `totbad`, and `avgErr` error metrics computed by the official evaluation tool. Table I shows results for the *Motorcycle* training pair at a 0.5px error threshold; the same trend held consistently across the additional image pairs we evaluated. AD-Census fusion reduced the bad-pixel rate (`bad0.5`) from 67.73% (Census-only) and 89.19% (AD-only) down to 59.30%, and reduced average endpoint error (`avgErr`) from 10.34px and 14.64px respectively down to 5.62px, which is roughly a 46% reduction in average error relative to the Census-only baseline. Notably, the fusion outperforms both of its constituent cost functions individually rather than simply averaging their behaviour, confirming that combining structural (Census) and intensity-based (AD) matching costs recovers information that neither term captures alone, consistent with the improvements reported by [15] for the original AD-Census formulation.

TABLE I
MIDDLEBURY MIDDLEVAL3 (*Motorcycle*, TRAINING, 0.5PX THRESHOLD)
ERROR METRICS FOR EACH COST FUNCTION.

Cost Fn	mask	bad0.5	invalid	totbad	avgErr
Census	86.6	67.73	1.17	68.91	10.34
AD	86.6	89.19	1.17	90.36	14.64
ADCensus	86.6	59.30	1.17	60.47	5.62

Hardware Performance. We evaluated the proposed hardware optimizations on the drone stereo dataset against the Vitis Vision Library implementation. Table III compares the disparity accuracy of the two implementations, while Table II reports their hardware resource utilization.

The final batched implementation processed 48 stereo frames with a total hardware pipeline time of 650.701 ms, corresponding to an average hardware latency of 13.556 ms per frame. Including data transfer overhead, the average end-to-end latency increased only slightly to approximately 13.642 ms per frame. The Vitis Vision implementation achieved an average hardware latency of 13.3 ms, meaning that the proposed implementation incurs only a 1.9% increase in hardware latency. Despite this small increase, the proposed design incorporates additional disparity processing and batched frame processing while maintaining hardware performance close to the highly optimized Vitis Vision baseline.

The final design used 73% of available BRAM, compared with 72% for the Vitis Vision baseline. DSP utilization increased from 20% to 62%, while FF utilization increased from 9% to 12% and LUT utilization increased from 60% to 89%. These increases reflect the additional hardware required for the increased disparity-level parallelism and batched processing. URAM utilization remained at 0% in the final implementation. Although the proposed design therefore uses substantially more compute and logic resources, it remains within the available resources of the KV260 while providing the additional accuracy and application-specific functionality required for the drone dataset.

Importantly, these hardware optimizations did not degrade disparity accuracy. The total bad-pixel rate decreased significantly from 17.28% for the Vitis Library implementation to 10.24% for the proposed hardware design for threshold of 1.0 pixels, and for threshold of 2.0px, the proportion of bad pixels decrease from 15.5% to 9.07%. Moreover, 90th percentile error for the original Vitis solution is 12.19, while it is only 4.78 for the improved solution - this is the case because the original code produces very bad outputs for some images, whereas the improved solution is reliably good across inputs. Compared to the Vitis design, average error decreased from 4.295px to 2.965 (Table III). II Demonstrates that the proposed optimizations achieved a equal latency and substantially improved memory efficiency while increasing disparity map accuracy on the drone dataset.

TABLE II
HARDWARE PERFORMANCE AND RESOURCE UTILIZATION ON THE DRONE DATASET. RESOURCE UTILIZATION IS REPORTED AS A PERCENTAGE.

Method	Latency	BRAM	DSP	FF	LUT	URAM
Vitis Vision	13.300 ms	72	20	9	60	0
Proposed (Batched)	13.556 ms	73	62	12	89	0

TABLE III
HARDWARE ACCURACY ON THE DRONE DATASET. BAD1.0 AND BAD2.0 STAND FOR PERCENTAGE OF PIXELS WITH ERROR OVER 1.0 AND 2.0 RESPECTIVELY. MEDERR STANDS FOR MEDIAN ERROR

Method	Bad1.0	Bad2.0	Avg Error	MedErr	90th percentile
Vitis Vision	17.28%	15.5%	4.295	0.62	12.19
Proposed	10.24%	9.07%	2.965	0.2485	4.78

Software-to-Hardware Speedup

Despite the similar hardware latency, the proposed implementation achieves a substantially larger speedup over its software baseline. The proposed hardware reduces the software execution time from 76967.8 ms to 13.556 ms, corresponding to an approximately 5680 $\times$ speedup. In comparison, the Vitis Vision implementation achieves an approximately 1501 $\times$ speedup over its software baseline. This difference reflects the

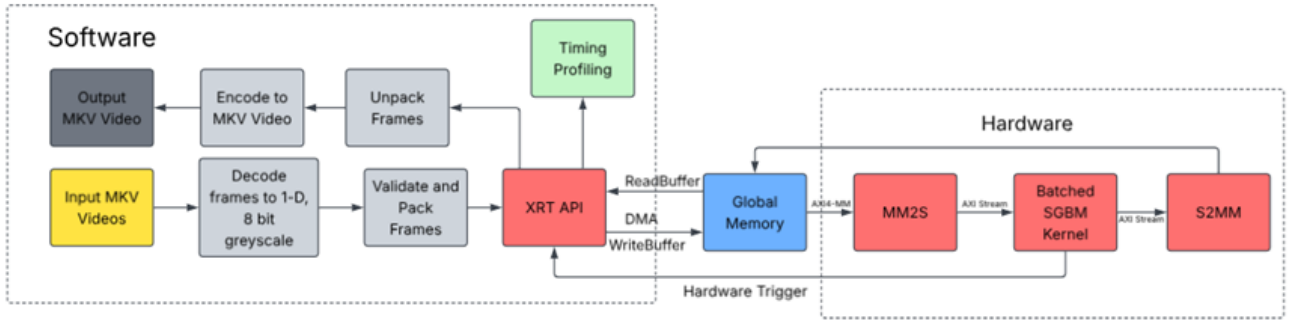


Fig. 5. Software-hardware execution flow for the SGBM accelerator on the KV260.

considerably higher execution time of the proposed software implementation rather than a substantially faster hardware accelerator.

Resolved Noise Problem As it was mentioned in the "Unused Approaches" section, during development we had an issue with noise which occurred when we combined all our modifications together. As we found out, the noise occurred because after the cost function was modified, its output ranged increased. However, the following SGBM optimization function assumed that the range stayed the same. This caused the compute disparity function to receive incorrect cost values from SGBM optimization and return incorrect outputs. Due to the noise, average error was 16.155, which is worse than baseline design accuracy, proportion of bad pixels was 31.41% for threshold of $1px$ and 30.03 for the threshold of $2px$. Moreover, 90th percentile error was 62.6x pixels, although median error was 0.37 pixels.

G. Qualitative Results

Figure 6 shows a qualitative comparison between the baseline Vitis Vision cost function and our AD-Census fusion on an indoor test scene, used here to isolate the effect of the cost fusion independently of the aerial dataset. The baseline disparity map (left) exhibits heavy salt-and-pepper noise across nearly all low-texture regions, including the wall and ceiling surfaces, with structural edges such as the roof beams and chair legs only weakly distinguishable from the surrounding noise. The AD-Census fusion result (right) shows a visible reduction in speckle noise on the wall, where the brick-like surface pattern becomes coherent rather than dissolving into random noise, and the chair and table outlines are better preserved with less noise bleeding into their boundaries. The ceiling region, however, remains heavily speckled in both cases, suggesting the fusion's benefit is concentrated in regions with some residual texture (e.g. the wall) rather than the near-featureless ceiling, where neither Census nor AD alone provides a strong enough matching signal.

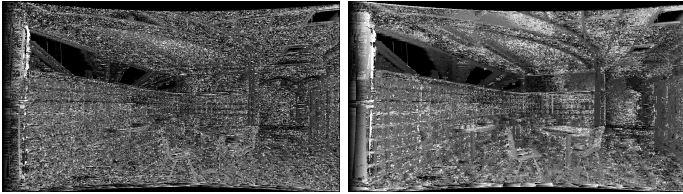


Fig. 6. Disparity maps for an indoor test scene: baseline Vitis Vision cost function (left) vs. AD-Census fusion (right).

Figure 7 shows the comparison between the noisy output as described in the "Resolved Noise Problem" section and disparity map for the input after the issue was resolved.

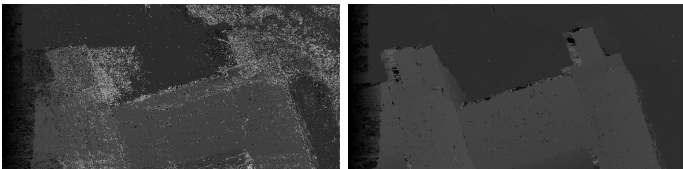


Fig. 7. Disparity map with salt-and-pepper noise (left) vs without (right)

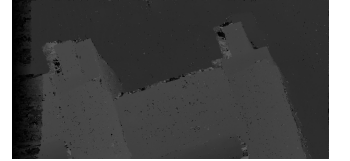
As it was mentioned before, the baseline Vitis solution has significant difference between good outputs and bad ones. Figure 8 shows a comparison between actual disparity map, output of the original algorithm in a bad case, and the output of the improved algorithm. We have noticed that the original algorithm performs worst with images that contain large buildings. This is the case because large buildings are closer to the camera and have larger disparity between images.



Ground Truth



Original



Proposed

Fig. 8. Comparison of disparity maps in a picture with a big building. The ground-truth disparity is shown at the top, followed by the output of the original algorithm and the proposed algorithm after resolving the cost-range mismatch.

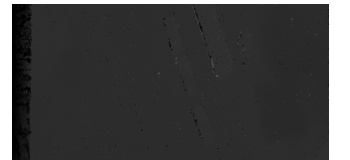
In other cases, the difference is less noticeable, as in Figure 9



Ground Truth



Original



Proposed

Fig. 9. Comparison of disparity maps in a picture without big buildings. The ground-truth disparity is shown at the top, followed by the output of the original algorithm and the proposed algorithm after resolving the cost-range mismatch.

VI. CONCLUSION

This project set out to profile and accelerate the Vitis Vision SGBM code on the Kria KV260. Profiling the baseline code identified that on-chip memory was the binding constraint, fixing the practical operating point at $NDISP=64$ and $PU=32$ before hardware linking failed. Rather than spending the resources recovered through our optimizations on

throughput, as FP-Stereo does on the ZCU102, we redirected them toward accuracy: higher input resolution, sub-pixel refinement, and a modified cost aggregation scheme suited to the large homogeneous regions typically seen in aerial imagery.

The results reflect this trade-off. Both the baseline and our design achieve the same order-of-magnitude speedup over their respective ARM software implementations (approximately $1501\times$ and $5680\times$), confirming that FPGA acceleration remains essential regardless of configuration. However, our hardware design does not outperform the baseline on raw latency, 13.3 ms as opposed to our 13.56 ms. What it delivers instead is a disparity map substantially closer to ground truth on the WHU MVS/Stereo drone dataset. These results validate the central premise of this work: on a resource-constrained device like the KV260, memory savings recovered through careful HLS restructuring are better spent on output quality than on speed, particularly for aerial applications where large textureless regions punish the baseline’s accuracy most.

REFERENCES

- [1] H. Hirschmuller, “Stereo processing by semiglobal matching and mutual information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008.
- [2] OpenCV, “Stereosgbm class reference,” Online, openCV 4.13.0 documentation. [Online]. Available: https://docs.opencv.org/4.13.0/d2/d85/classcv_1_1StereoSGBM.html
- [3] J. Zhao, T. Liang, L. Feng, W. Ding, S. Sinha, W. Zhang, and S. Shen, “Fp-stereo: Hardware-efficient stereo vision for embedded applications,” in *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*, 2020, pp. 269–276.
- [4] C. Banz, S. Hesselbarth, H. Flatt, and H. Blume, “Real-time stereo vision system using semi-global matching disparity estimation: Architecture and fpga-implementation,” in *2010 International Conference on Embedded Computer Systems (SAMOS)*, Aug. 2010.
- [5] J. Kalomiros, J. Vourvoulakis, and S. Vologianidis, “A hardware accelerator for the semi-global matching stereo algorithm: An efficient implementation for the Stratix V and Zynq UltraScale+ FPGA technology,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 1, pp. 1–25, 2024.
- [6] S. Lahti and T. D. Hämmäläinen, “High-level synthesis for FPGAs—a hardware engineer’s perspective,” *IEEE Access*, vol. 13, pp. 28 574–28 593, 2025.
- [7] Advanced Micro Devices, “Vitis vision library,” https://github.com/Xilinx/Vitis_Libraries/tree/2025.1, 2025, version 2025.1, accessed 16 August 2026.
- [8] Xilinx, *ZCU102 Evaluation Board User Guide*, Xilinx, Inc., Feb. 2023, uG1182 (v1.7).
- [9] Advanced Micro Devices, *Kria KV260 Vision AI Starter Kit Data Sheet*, Advanced Micro Devices, Inc., Jun. 2025, dS986 (v1.3).
- [10] R. A. Hamzah and H. Ibrahim, “Literature survey on stereo vision disparity map algorithms,” *Journal of Sensors*, vol. 2016, p. 8742920, 2016.
- [11] J. Liu and S. Ji, “A novel recurrent encoder-decoder structure for large-scale multi-view stereo reconstruction from an open aerial dataset,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6050–6059.
- [12] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs*, 2024th ed., AMD, Jun. 2024. [Online]. Available: https://docs.amd.com/access/sources/dita/map?Doc_Version=2024.1+English&url=ug949-vivado-design-methodology
- [13] J. Liu and S. Ji, “A novel recurrent encoder-decoder structure for large-scale multi-view stereo reconstruction from an open aerial dataset,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6049–6058.
- [14] Middlebury Stereo Evaluation, “Middlebury stereo evaluation - version 3,” <https://vision.middlebury.edu/stereo/eval3/>, accessed: 2026-08-17.
- [15] X. Mei, X. Sun, M. Zhou, S. Jiao, H. Wang, and X. Zhang, “On building an accurate stereo matching system on graphics hardware,” in *2011 IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*, 2011, pp. 467–474.